

Animations

Mr. Tan Chiang Kang



Animations in Mobile

- A few types of animations that can be achieved in mobile applications:
 - Animated GIF images
 - Frame-by-frame animation
 - Tweened animation
 - Importing external animation

Animated GIF images

- Animated GIF is a type of animated image file
- It was formed by a series of GIF images
- When frames are played in a sequence through time, it will form movements (animation)
- In Android, the Animated GIF could be imported as a drawable resource
- However interpretation of animated GIF on Android is not always convenient and may require decoding

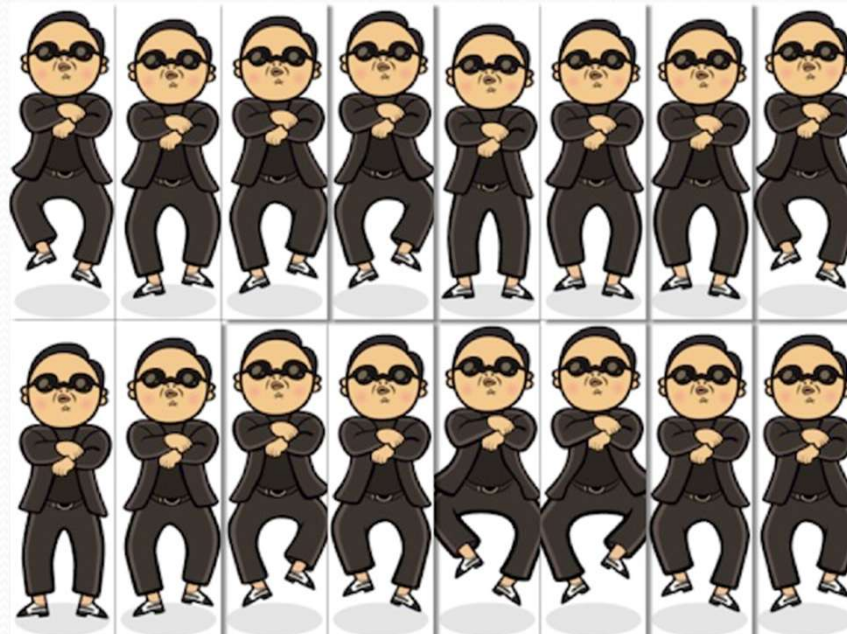
Animated GIF

- An example of an animated GIF using 3 frames



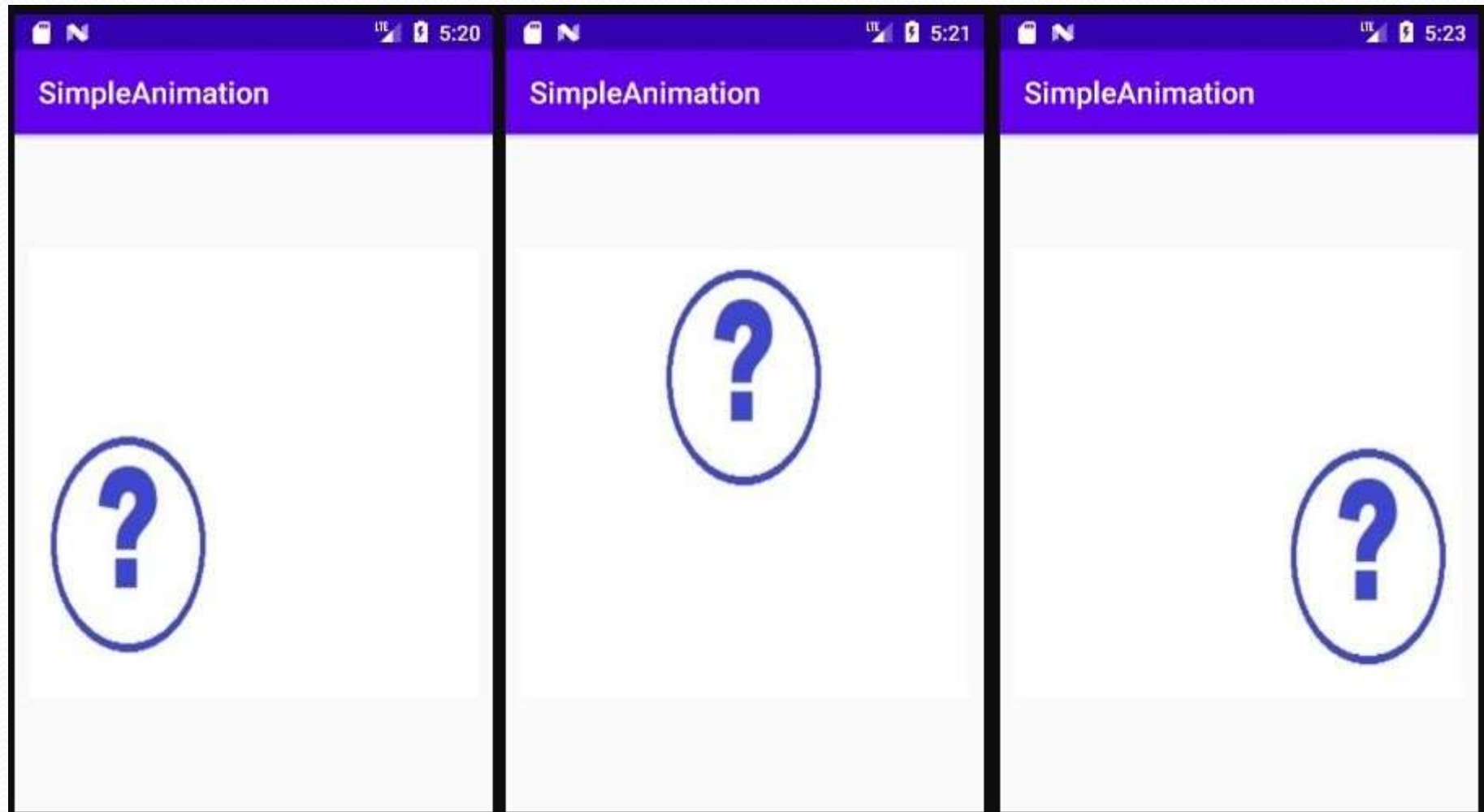
Frame-by-frame Animation

- Animation/video sequences are achieved through chaining a series of frames (e.g. Animated GIF)
- Therefore if the source frames are available, animation could be recreated during runtime



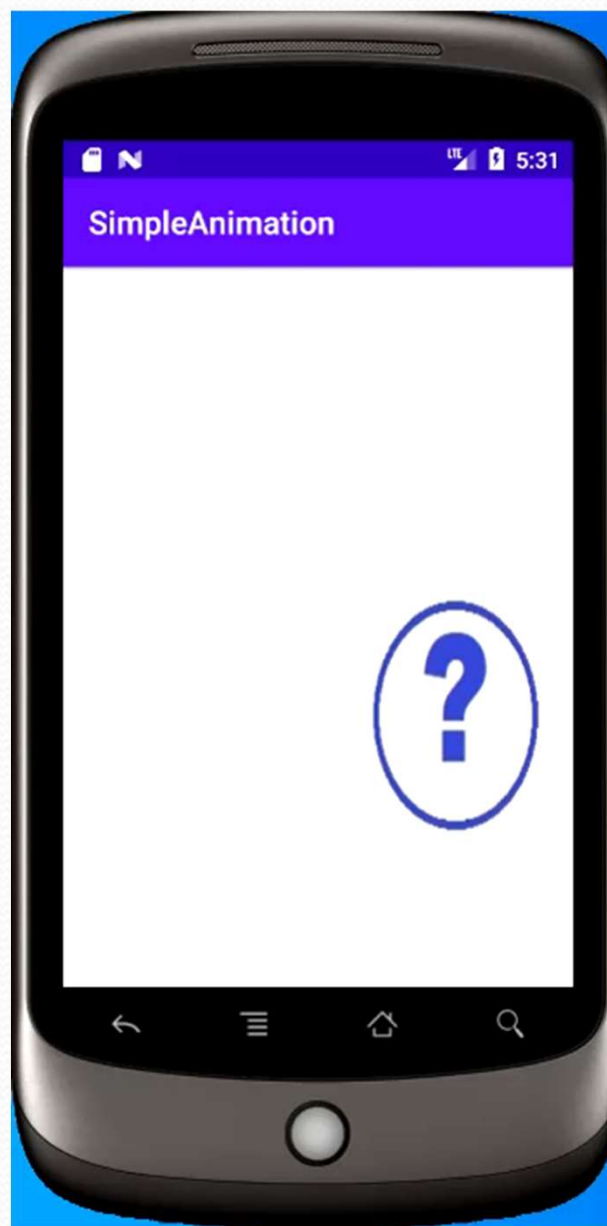
Frame-by-frame Animation

- Each frame can be created as a **BitmapDrawable**
- Each frame is added into an **AnimationDrawable** based on a certain duration
- The frames should be of the same size
- The animation can be displayed using an **ImageView**
- In order to allow repetitive loop for the animation, we can use **setOneShot()**
- The animation can be triggered to start using **start()**
- The animation can be stopped at any time using **stop()**



Android Developers guide on how to animate drawable graphics:

<https://developer.android.com/guide/topics/graphics/drawable-animation#java>





Tweened Animation

- Frame-by-frame animation requires all frames to be created as different images
- The process is haptic and takes up much space
- Tweened animation allows animation to be rendered by the system
- Such animation can often be applied on any UI element
- Sample animations include: rotation, fading, movement, resize, etc



Tweened Animation in Android

- Tweened animation can be applied on any type of **View** (e.g. **ImageView**)
- We can listen for **Animation** events, including the start, end and repeat events
- Achieved through the implementation of **AnimationListener**

Animation Listener - codes

- Set the listener on the Animation object:

```
an.setAnimationListener(new MyListener());
```

```
Class MyListener implements Animation.AnimationListener{  
    public void onAnimationEnd(Animation animation) {  
        //Do at end of animation  
    }  
  
    public void onAnimationRepeat(Animation animation){  
        //Do each time the animation loops  
    }  
  
    public void onAnimationStart(Animation animation){  
        //Do at start of animation  
    }  
}
```

Movement (Translate)

- Move an object to another location along a path
- Moves from starting position (**fromXDelta**, **fromYDelta**) to relative target location (**toXDelta**, **toYDelta**)
- The direction of movement are defined by the signs of the values of the target location
 - Positive values towards right and downward
 - Negative values towards left and upward

Transparency Changes (Alpha)

- The transparency is controlled using Alpha transformations
- Alpha transformations can be used to fade objects in and out of view, or to layer them on the screen
- Alpha values range from 0.0 (invisible) to 1.0 (solid)
- The changes are defined from the starting (**fromAlpha**) to the end (**toAlpha**)
- To create Alpha transformation in programming, **AlphaAnimation** is used

Rotations (Rotate)

- Used to spin objects around a pivot point
- Rotations are defined in terms of degrees
- The rotations start with **fromDegrees** until **toDegrees**
- The positive **toDegrees** would allow the object to rotate clockwise while negative values for counterclockwise
- Other than that, the pivot point can also be set through **pivotX** and **pivotY**, where the rotation will be based on

Scaling (Scale)

- Scaling can be used to enlarge or shrink an object vertically or horizontally
- Scaling are operating based on relative scales
- Scale value of 1.0 refers to 100 percent
- The scales are defined by starting scale (**fromXScale, fromYScale**) to target scale (**toXScale,toYScale**)
- The pivot point can also be set for using **pivotX** and **pivotY**

Fillafter Property

- In order to avoid the object to jump back to the original point after the animations, **fillAfter** property can be included and set to true
- By default, if this property is not set, the object will revert back to the original position and scale in a blink

External Animations

- In order to produce more complicated animated graphics (esp. in games, 3D and etc), external tools can be used to produce animated graphics
- Many such tools are available and can be exported to various mobile development platforms, enabling cross-platform development
- For example **Unity**



Multi-threading

- When certain animation are meant to be used in the background, it is recommended to run it on a separate thread
- Examples are using **ImageSwitcher** to change and display **Drawable** resource using a timer

What's Next?

- Storing data persistently

Storing Data in Mobile Devices

- Mobile devices generally have restricted data storage
- In Java ME dominant era, file system in mobile devices are not accessible
- There are also serious restriction in storing data for the Recordstore Management System (RMS) that is used by Java ME
- Back then, data can only be stored as a list of entries with one single data stored in byte arrays
- Query of data was complicated and required to be coded

Modern Storage

- Era had changed, storage of phones are much more powerful today, allowing more possibilities:
 - A proper relational database on mobile devices
 - Simple data pairs
 - Files
 - Specialised data (contact list, calender item, etc)
 - Cloud (outside the phone)

SQLite Database

- Database systems in the desktop environment are often pretty resource consuming
- The database system must be small in order that it can be used in storage-restricted devices
- SQLite is a database designed to be lightweight and file-based, making it ideal for embedded devices



SQLite Database in Android

- The database created using SQLite is however private to applications and not shared
- The Android SDK provides useful classes to handle SQLite database management
- Can be easily used by using **android.database.sqlite** packages

Creating SQLite Database

- There are various ways to create databases on Android
- One of the method is to create the database using **openOrCreateDatabase()** method
- ```
SQLiteDatabase db =
 openOrCreateDatabase(
 "our_database.db",
 SQLiteDatabase.CREATE_IF_NECESSARY,
 null);
```

# Using SQLiteOpenHelper

- Another method is to use the **SQLiteOpenHelper** class
- A subclass for **SQLiteOpenHelper** is required to be defined and the constructor will be used to create the database
- The **SQLiteOpenHelper** also includes **onCreate()** and **onUpgrade()** methods to help create and update tables

# Database in File System

- The created database will be stored at  
/data/data/<application package name>/databases/  
<databasename>
- For example:  
/data/data/my.edu.utar.sqlproject/databases/  
our\_database.db

# SQLite Database Properties

- If you create the database instance without using SQLiteOpenHelper, you can set the properties using a series of set methods
- Important properties to be set includes version, locale, thread-safe locking feature
- For example:

```
db.setLocale(Locale.getDefault());
db.setLockingEnabled(true);
db.setVersion(1);
```

# Creating Tables

- To create new tables, they can be done by defining the SQL statement for creating a table

- For example:

```
CREATE TABLE our_table (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 Content1 TEXT,
 Content2 TEXT);
```

- It can be stored as a **String** but the spaces must be carefully taken care of

# Executing SQL

- The execution of the table creation can be triggered by calling **execSQL()** method
- For example the SQL statement is stored as  
CREATE\_OUR\_TABLE  
`db.execSQL (CREATE_OUR_TABLE) ;`
- The **execSQL()** method can also be used to execute other SQL statements, e.g. to update and delete tables, view, trigger and etc

# Foreign Key Constraints

- Foreign key constraints are used to enforce relationships (if there is one) between tables.

```
CREATE TABLE artist(
 artistid INTEGER PRIMARY KEY,
 artistname TEXT
);
CREATE TABLE track(
 trackid INTEGER,
 trackname TEXT,
 trackartist INTEGER
);
```

[sqlite.org/foreignkeys.html](http://sqlite.org/foreignkeys.html)

- Both tables, *artist* and (music) *track*, have something in common, i.e. the artist ID.
- What problems can you foresee if the two tables are maintained independently?

# Foreign Key Constraints

- Solution: Add an SQL foreign key constraint to the database schema to enforce the relationship between the *artist* and *track* table.

```
CREATE TABLE track(
 trackid INTEGER,
 trackname TEXT,
 trackartist INTEGER,
 FOREIGN KEY(trackartist) REFERENCES artist(artistid)
);
```

[sqlite.org/foreignkeys.html](http://sqlite.org/foreignkeys.html)

- One exception, if the foreign key is null in a *track*, no corresponding entry in the *artist* table is required.

# Foreign Key Constraints

- SQLite did not support foreign key constraints prior to Android Froyo (API level 8) or SQLite version 3.6.19
- Even when it is supported, SQLite does not enforce foreign keys by default
  - To enable it, you can do so in your custom **SQLiteOpenHelper** class, under **onConfigure()** method

```
@Override
public void onConfigure(SQLiteDatabase db) {
 super.onConfigure(db);
 db.setForeignKeyConstraintsEnabled(true);
}
```

# Inserting Records

- To insert a new record into the database, we can use a method **insert()** to accomplish the task from the **SQLiteDatabase** class

```
ContentValues values = new
 ContentValues();
values.put("content1", "data");
values.put("content2", "data2");
long newId = db.insert("our_table", null,
 values);
```

# Updating Records

- To modify existing records, we can use **update()** method
- It takes 4 arguments:
  - The table to update records
  - ContentValues object with updated information
  - WHERE clause (**OPTIONAL**)
  - An array of WHERE clause arguments (**OPTIONAL**)
- If WHERE clause is set to be **null**, changes will apply on all records

# Updating Records

- Only the field that requires update needs to be stated in the ContentValues
- Updates will occur according to the conditions specified WHERE clause

```
ContentValues values = new
ContentValues();
values.put("content2", "newdata");
db.update("our_table", values, "id=?",
 new String[] {"5"});
```

# Deleting Records

- To delete a record, the **delete ()** method is used
- It takes 3 arguments:
  - The table involved
  - WHERE clause (OPTIONAL)
  - Array of WHERE clause arguments (OPTIONAL)
- If the WHERE clause is **null**, all records will be deleted from the table

# Deleting Records

- Removes all records:

```
db.delete("our_table", null, null);
```

- Removes records using primary key:

```
db.delete("our_table", "id=?",
 new String[] {"4"});
```

- Removes records based on other criterias:

```
db.delete("our_table", "contents1=?",
 new String[] {"somedata"});
```

# Transactions

- Often you may need to run multiple database operations at one run
- SQL Transactions help to group operations together
- If any of the operations fail, you can handle the error and either recover or rollback all operations
- The transaction is only committed if all operations are successful

# Transactions - structure

- Sample structure of a transaction:

```
db.beginTransaction();
try{
 // Insert some records, updated others, delete a few
 // Do all necessary operations, then commit it

 db.setTransactionSuccessful();
} catch (Exception e) {
 // Transaction failed. Do something here.
} finally {
 db.endTransaction();
}
```

# Transactions

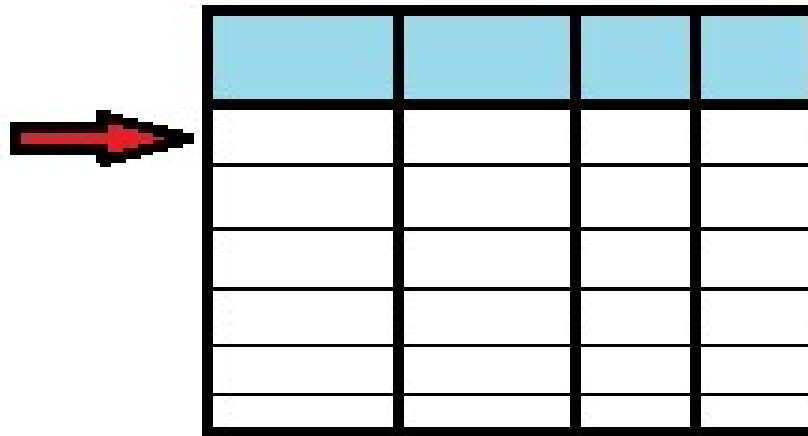
- The transaction will begin after the called of **beginTransaction()** method
- The transaction remains open in a try/catch block
- If operations are okay, **setTransactionSuccessful()** method must be triggered to commit a transaction
- An **endTransaction()** method is called to complete the transaction
- If there is any error in the operations, you do not need to explicitly rollback - **endTransaction()** without **setTransactionSuccessful()** will lead to a rollback automatically.

# Database Query

- The power of a database is when we can extract the data from it according to our needs
- A query can be executed to extract the required data
- SQLiteDatabase allows the user to use SQL queries and hence it is pretty convenient compared to Java RMS's way of query
- A query could be executed in various ways, including the direct use of SQL statements or more popularly, using the **Cursor** class

# Cursors

- **Cursor** is used to access results returned from the SQL query
- They act like file pointers, allowing random access
- Import `android.database.Cursor` class



# Query Parameters

- Parameters for **query()** method:
  - Optional Boolean that specifies if the result set should be unique
  - Name of table to query
  - A projection, as an array of strings, that list the columns to include in the result set
  - WHERE clause that defines the rows to be returned
  - Array of selection arguments of the WHERE clause

# Query Parameters

- Parameters for **query()** method:
  - A GROUP BY clause that defines how resulting rows are grouped
  - A HAVING filter that defines which row groups to include for the GROUP BY clause
  - A string that describes the order of the returned rows
  - Optional string that defines a limit for the number of returned rows

# Cursor – example 1

- A simple example of query with only table included will be a simple query of all data from the table

```
//SELECT * FROM our_table
Cursor c = db.query("our_table", null,
 null, null, null, null, null);
c.close();
```

## Cursor – example 2

- WHERE clause can be used by setting the field to check follow by arguments
- The use of “?” will allow an array of arguments to be included

```
//SELECT * FROM our_table WHERE id=9;
Cursor c = db.query("our_table", null,
 "id=?", new String [] {"9"}, null,
 null, null);
```

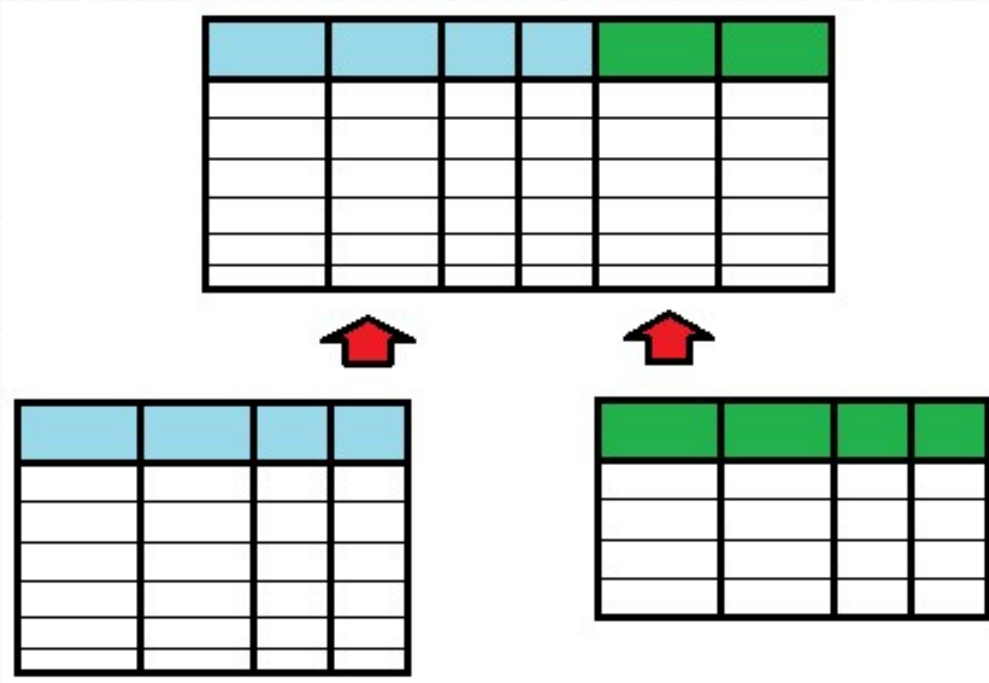
## Cursor – example 3

```
//SELECT content1, id FROM our_table
ORDER BY content1 ASC;
String columnsToReturn = {"content1",
"id"};
String sortOrder = "content1 ASC";

Cursor c = db.query("our_table",
columnsToReturn, null, null, null,
null, sortOrder);
```

# SQLiteQueryBuilder

- To work with more complex queries that involved more than one table, the **SQLiteQueryBuilder** class is used



# SQL statement – INNER JOIN

```
SELECT our_table.id,
our_tablescontent1,
another_table.field1,
another_table.field2,
our_table.anotherid
FROM our_table
INNER JOIN another_table on
our_table.anotherid=another_table.i
d
ORDER BY content1 ASC;
```

# SQLiteQueryBuilder – example

```
SQLiteQueryBuilder queryBuilder = new
 SQLiteQueryBuilder();
queryBuilder.setTables("our_table",
 "another_table");
queryBuilder.appendWhere(
 "our_table.anotherid=another_table.id");

String asColumnsToReturn[] = {"our_table.id",
 "our_tablescontent1", "another_table.field1",
 "another_table.field2", "our_table.anotherid" };
String sortOrder = "content1 ASC";

Cursor c = queryBuilder.query(db, asColumnsToReturn,
 null, null, null, null, sortOrder);
```

# Raw Queries

- It is possible to directly use the SQL statements without using the **query()** methods provided
- This can be accomplished by calling **rawQuery()** method and passing the SQL statement in as a string

# Raw Queries – SQL example

```
SELECT content1 AS Info,
'our_table' AS OriginalTable
FROM our_table
WHERE Info LIKE '%ow%'
UNION
SELECT (field1 || ' ' || field2) AS Info,
'another_table' AS OriginalTable
FROM another_table
WHERE Info LIKE '%ow%'
ORDER BY Info ASC
```

# Raw Queries – example

```
String sqlUnionExample = "SELECT content1
 AS Info, 'our_table' AS OriginalTable
 FROM our_table WHERE Info LIKE ? UNION
 SELECT (field1 || ' ' || field2) AS Info,
 'another_table' AS OriginalTable FROM
 another_table WHERE Info LIKE ? ORDER BY
 Info ASC; ";
```

```
Cursor s = db.rawQuery(sqlUnionExample,
 new String[] { "%ow%", "%ow%" });
```

# Managing Cursor

- When a **Cursor** is intensive, with too many returned records, it is best to separate it in a different UI thread
- **Cursor** needs to be managed as part of the application lifecycle
- **Cursor** should be deactivated when the application pauses or shut down – **deactivation()**
- **Cursor** should refresh its data when the application restarts – **requery()**
- **Cursor** should be closed when its no longer used – **close ()**

# Managing Cursor

- Such management calls can be done accordingly within the application lifecycle through callbacks such as **onPause()**, **onResume()** and **onDestroy()**
- **Cursor** can also be managed automatically without needing developers to plan on all the procedures
- The automatic way is to allow the parent **Activity** to manage it by calling **startManagingCursor()**
- To exit automatic management and returns to manual management, simply call **stopManagingCursor()**

# Extracting from Cursor

- After reading the data into the **Cursor**, we can iterate among the **Cursor** to read the data out
- We can logically read from the beginning of the **Cursor** and move to the next records until the end of the **Cursor** to read all extracted records

# Cursor Methods

- Some Cursor class navigation methods as follows:
  - **moveToFirst** – moves the cursor to the first row in the query results
  - **moveToNext** – moves the cursor to the next row
  - **moveToPrevious** – moves the cursor to the previous row
  - **getCount** – returns the number of rows in the result set
  - **getColumnIndexOrThrow** – returns the index for the column with the specified name (throwing exception if column does not exist)

# Cursor Methods

- Some Cursor class navigation methods as follows:
  - **getColumnName** – returns the name of the specified column index
  - **getColumnNames** – returns a string array of all the column names in the current Cursor
  - **moveToPosition** – moves the Cursor to the specified row
  - **getPosition** – returns the current Cursor position
  - **isAfterLast** – check if the Cursor had moved beyond the last record

# Query Extraction - example

```
c.moveToFirst();
while (c.isAfterLast() == false) {
 String results = " | ";
 for (int i = 0; i < c.getColumnCount();
 i++) {
 results = results.concat(
 c.getString(i) + " | ");
 }
 c.moveToNext();
}
```

# Deleting Tables

- Deleting tables can be easily achieved using SQL statements
- For example:  
`db.execSQL("DROP TABLE our_table;");`
- The same idea could also be applied to delete other SQLite objects, such as triggers

- For example:  
`db.execSQL("DROP TRIGGER IF EXISTS  
fk_content1;");`

# Closing Database

- The database should be closed when they are not in use using the **close()** method
- A closed database can be reopened and remains in the storage

```
db.close();
```

# Deleting Database

- When a database is no longer required, it could be permanently deleted
- To delete a database, a **deleteDatabase()** method could be used
- A database is deleted by providing the name of the database, unlike other procedures, that are triggered from the database object

```
deleteDatabase("our_database.db");
```

# SQLiteOpenHelper

- The database usually survives throughout the lifetime of the application from its first installed until it is removed
- The database is rarely deleted or removed in the middle of the process
- **SQLiteOpenHelper** helps to manage the creation of database and tables

# Example – SQLiteAdapter.java

```
public class SQLiteHelper extends SQLiteOpenHelper {
 public SQLiteHelper(Context context, String name,
 CursorFactory factory, int version) {
 super(context, name, factory, version);
 }

 @Override
 public void onCreate(SQLiteDatabase db) {
 //run scripts during creation of DB
 }

 @Override
 public void onUpgrade(SQLiteDatabase db, int oldVersion, int
 newVersion) {
 //run this if there is an update in DB version
 }
}
```

# Using SQLiteOpenHelper Class

- The SQLiteOpenHelper class helps to manage a database and will create or upgrade the database when required
- For the **onCreate** method, it is called once only when the application is first installed, the developer can create the database, tables and initialise the data here
- For the **onUpgrade** method, it will be called only if the application is installed with a database version that is higher than current

# Using SQLiteOpenHelper Class

- This practice helps to ensure the data of the database is not deleted during an installation of a newer version of the application
- Necessary upgrades such as adding tables, adding data and etc can be achieved
- The developer should carefully plan on the necessary upgrades based on version (e.g. upgrade from ver 1 to ver 4 likely differs from upgrade from ver 3 to ver 4)

# Using SQLiteOpenHelper Class

- Customised methods can be created to then access the created database including data entry, query and etc
- The **Activity** will use these methods to work with the database usually rather than directly working with the database itself
- The database will then only be accessible by the **Activity** itself

# What's Next?

- Other storage options